

Image Classification of Brain Tumors

Project Report

1. Introduction

Detecting the presence of tumors early and accurately is really important in medicine, because it can have a big impact on treatment choices and patient outcomes. Doctors often examine medical images to determine whether a tumor is present, but this process can be time-consuming and challenging due to subtle variations in image appearance. Machine learning, especially image classification, can help by providing automated and more consistent analysis of tumor images [1].

The report is structured as follows: Section 1 introduces the task and motivation. Section 2 formulates the problem, describing the data, labels, and setup. Section 3 presents the methodology, including preprocessing, the CNN model, loss and optimizer choices, and data splits. Section 4 discusses the results of the experiments, followed by Section 5, which compares the models and explains the final selection. Section 6 concludes the project, and the final part briefly describes the use of AI tools during the work.

2. Problem formulation

Our application can be formalized as a supervised binary classification problem. The goal is to classify MRI images of brains as either containing a tumor or not. Each MRI scan works as a data point, and the features are the pixel values of the image. To make sense of these pixels and find useful patterns, we use Convolutional Neural Networks (CNNs).

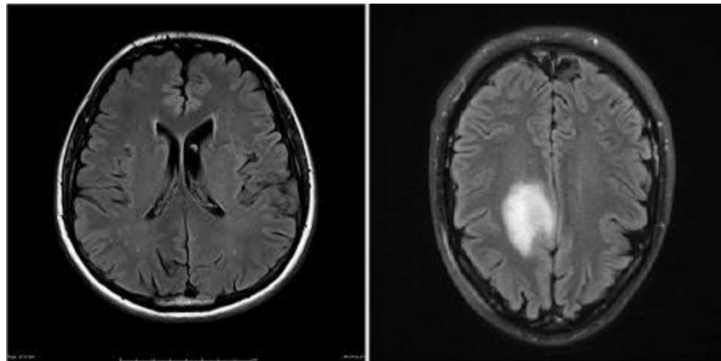


Figure 1: Example of a brain MRI scan with and without a tumor from the dataset.

The dataset we are using for this project consists of MRI (Magnetic Resonance Imaging) scans, which are images of brains with and without tumors. The images are publicly available on Kaggle, provided by Preetviradiya [2]. It is a simple binary dataset with two classes: "Brain Tumor" and "Healthy". The label for each image is binary, meaning it just tells us if a tumor is present or not.

3. Methods

We started with splitting the dataset into training, validation, and test sets using a 70% / 20% / 10% ratio. The training set has 2343 brain tumor images and 2066 healthy images, the validation set has 1406 brain

tumor images and 1237 healthy images, and the test set has 811 brain tumor images and 724 healthy images. We chose this ratio for splitting the dataset because it is commonly used in machine learning projects and provides a good balance between training, validation, and testing. The dataset is relatively balanced, with only about a 6% difference between the number of samples in each class. This small imbalance should not pose a problem for our experiments.

The images in our dataset come in different sizes (Figure 1), so as part of preprocessing we resized them to 128×128 pixels using TensorFlow’s ImageDataGenerator [3]. For training and validation, we also normalized the images to $[0,1]$ and applied simple augmentations like horizontal flips, small zooms, and slight shearing to make the model more robust. The test images were only normalized to allow fair evaluation. All images were treated as RGB, categorized for binary classification (tumor vs. healthy), and the training set was shuffled to help generalization.

We experimented with two models: a simple linear regression model and a Convolutional Neural Network (CNN). The linear regression model was chosen as a baseline due to its simplicity and interpretability, providing a clear reference point for how a non-hierarchical model performs on this task. It is also suitable for binary classification problems, as it can separate two classes using a linear decision boundary. The CNN was selected because it is specifically designed to process visual data and can automatically learn spatial hierarchies of features, such as edges, textures, and shapes, which are essential for recognizing patterns in medical images.

As our loss function, we chose binary cross-entropy, since our task is a binary classification problem – the model predicts whether a brain MRI contains a tumor or not. Binary cross-entropy measures how close the predicted probabilities are to the true labels (0 or 1) and is well-suited for this kind of task.

For optimization, we used the Adam optimizer with a learning rate of 0.001. This optimizer is convenient because it automatically adjusts the learning rate for each parameter, which usually helps the model learn faster and more reliably than regular gradient descent.

4. Results

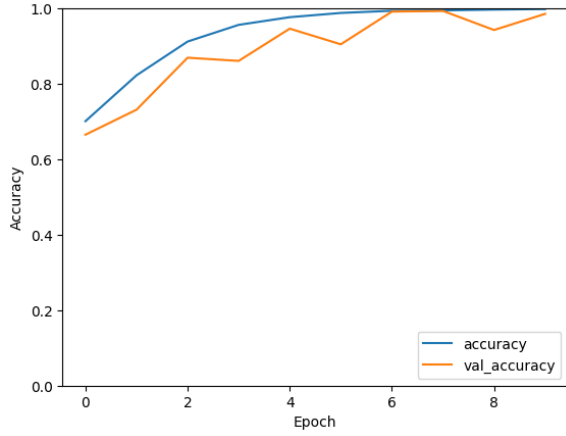
This section compares the performance of the two models trained for binary tumor classification: (1) a Convolution Neural Network (CNN) and (2) a simple linear Artificial Neural Network (ANN).

The CNN model consisted of two convolutional layers (with 32 and 64 filters, respectively), each followed by a MaxPooling layer. The convolutional base was flattened and passed through two fully connected layers with ReLU activation, batch normalization, and dropout for regularization, before reaching the final sigmoid output layer. The model was trained for 10 epochs using the Adam optimizer with a learning rate of 0.001.

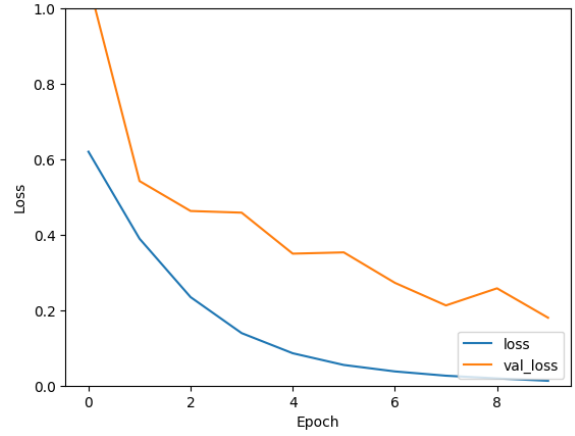
At the end of training, the model achieved a **training accuracy of 99.88%**, a **validation accuracy of 98.56%**, and a **testing accuracy of 98.49%**. The corresponding losses were 0.0134 (training), 0.1800 (validation), and 0.1954 (testing).

Table 1: Summary of training, validation and test results (loss).

Model	Training Loss	Validation Loss	Test Loss
CNN	0.0134	0.1800	0.1954



(a) CNN: accuracy vs. epochs



(b) CNN: loss vs. epochs

Figure 2: Training/validation curves for the CNN model

The second model was a simple feedforward ANN that flattened the input image and passed it directly to a sigmoid output neuron, effectively functioning as a linear classifier over pixel intensities.

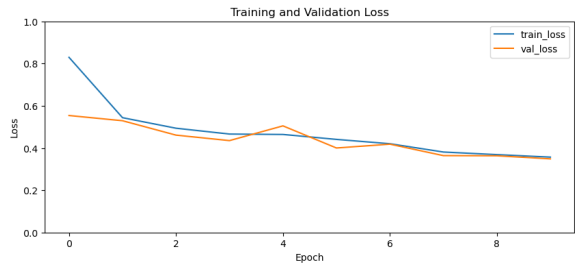
After 10 epochs, the ANN achieved a **training accuracy of 87.83%**, a **validation accuracy of 88.52%**, and a **testing accuracy of 87.74%**. The losses were 0.357 (training), 0.3488 (validation), and 0.3481 (testing).

Table 2: Summary of training, validation and test results (loss).

Model	Training Loss	Validation Loss	Test Loss
Linear ANN	0.3570	0.3488	0.3481



(a) ANN: accuracy vs. epochs



(b) ANN: loss vs. epochs

Figure 3: Training/validation curves for the Linear ANN model.

5. Model Comparison and Final Selection

Comparing both architectures, it is evident that the CNN model significantly outperformed the linear ANN in terms of accuracy, loss, and overall generalization. The CNN model's ability to capture spatial hierarchies and local features allowed it to detect subtle differences in brain MRI images, which the linear ANN failed to model effectively.

The final chosen method is therefore the **CNN architecture**, due to its superior accuracy, lower error, and better generalization across all datasets. The **test error** of the final chosen method (CNN) is

approximately **1.51%**, confirming its reliability for tumor classification.

To further assess the CNN's performance, we tabulated the precision-recall-F1 summary.

CNN Model - Classification Report					Linear ANN - Classification Report				
precision	0.98	1.00	0.99	0.99	precision	0.74	0.93	0.80	0.84
recall	1.00	0.98	0.99	0.99	recall	0.96	0.60	0.80	0.78
f1-score	0.99	0.99	0.99	0.99	f1-score	0.84	0.73	0.80	0.78
support	477.00	404.00	0.99	881.00	support	477.00	404.00	0.80	881.00
	Tumor	Healthy	accuracy	macro avg		Tumor	Healthy	accuracy	macro avg

Figure 4: Classification report comparison between CNN and Linear ANN models.

Linear Model		
True label	Tumor	1297
	Healthy	171
	Tumor	156
	Healthy	1043
	Predicted label	

CNN Model		
True label	Tumor	1467
	Healthy	1
	Tumor	16
	Healthy	1183
	Predicted label	

Figure 5: Confusion Matrix comparison of both models

6. Conclusion

This project demonstrated the successful application of deep learning to the problem of brain tumor classification from MRI images. The CNN achieved nearly 98.5% accuracy on the test set, suggesting that the model has learned to generalize well to unseen data. These results indicate that the problem has been solved satisfactorily for the current dataset. However, there remains room for improvement. The model was trained for only 10 epochs; extending training with early stopping or learning rate scheduling might yield even better generalization.

Limitations include the relatively small dataset size and potential overfitting despite dropout and normalization. Moreover, since medical imaging can vary across scanners and patient populations, testing the model on more diverse datasets would strengthen its reliability for real-world deployment.

References

- [1] AB Abdusalomov, M Mukhiddinov, and TK Whangbo. Brain tumor detection based on deep learning approaches and magnetic resonance imaging. *Cancers (Basel)*, 15(16):4172, Aug 18 2023.
- [2] Preetviradiya. Brain tumor dataset. <https://www.kaggle.com/datasets/preetviradiya/brian-tumor-dataset>, 2021. Accessed: 2025-09-16.
- [3] TensorFlow Authors. Imagedatagenerator. https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator, 2023. Accessed: 2025-09-17.